

Personal Study Material

Learner: **STU001** · 07 October 2026

Your knowledge state

Concept	Mastery	Level	Status
String Manipulation and Text Processing	50%	Beginner	Study this
Operators, Pointers and Control Structures	51%	Intermediate	Study this
Fundamentals and System Architecture	58%	Intermediate	Study this
Data Structures, Memory Management, and File Handling	63%	Intermediate	Study this
Algorithmic Problem Solving and Array Manipulation	74%	Expert	On target

A concept needs study when mastery is below the course target of 70% (the pass threshold is 50%).

How to use this material

1. Start with Priority 1, your weakest concept.
2. Look at the questions you got wrong and find out why the right answer is right.
3. Read the topics to revise and the explanation, then the syllabus pages that follow them.
4. Do the practice activities without looking at the notes.
5. Tick the self-check list, then take the adaptive test in the app.

Priority 1: String Manipulation and Text Processing

CO4 · Mastery 50% · exam score 0% · level Beginner

Questions you got wrong

- ✗ Which header file declares strlen, strcpy and strcmp? (easy)
- ✗ How is the end of a string marked in C? (easy)
- ✗ Write a function that counts the vowels in a string. (medium)

1. What you will be able to do

- Explain the key ideas of String Manipulation and Text Processing.
- Apply characters correctly in small programs.
- Identify common mistakes involving strcmp.

2. Topics to revise

1. characters

- The functions in ctype.h such as toupper, tolower, isalpha and isdigit classify and convert individual characters.
- Counting vowels in a string is done by traversing the characters until the null character and testing each one.

2. strcmp

- The functions strlen, strcpy, strcat and strcmp are declared in the header file string.h.
- Comparing two strings with == compares their addresses, so strcmp must be used to compare their contents.

3. compares

- A palindrome check compares characters from both ends of the string moving towards the middle.

4. buffer

- Copying a string into a buffer that is too small causes a buffer overflow, so the destination must have room for the null character.

3. Explanation

The functions strlen, strcpy, strcat and strcmp are declared in the header file string.h. Comparing two strings with == compares their addresses, so strcmp must be used to compare their contents. Copying a string into a buffer that is too small causes a buffer overflow, so the destination must have room for the null character. The functions in ctype.h such as toupper, tolower, isalpha and isdigit classify and convert individual characters.

Reading material from your syllabus

These are the parts of the course notes that cover this concept. Read them before you try the practice activities.

Unit 4: String Manipulation and Text Processing

A string in C is an array of characters terminated by the null character '\0'.

The functions `strlen`, `strcpy`, `strcat` and `strcmp` are declared in the header file `string.h`.

`strlen` returns the number of characters before the terminating null character.

Comparing two strings with `==` compares their addresses, so `strcmp` must be used to compare their contents.

`strcmp` returns zero when two strings are equal, a negative value when the first is smaller and a positive value otherwise.

Copying a string into a buffer that is too small causes a buffer overflow, so the destination must have room for the null character.

The functions in `ctype.h` such as `toupper`, `tolower`, `isalpha` and `isdigit` classify and convert individual characters.

Counting vowels in a string is done by traversing the characters until the null character and testing each one.

A palindrome check compares characters from both ends of the string moving towards the middle.

Counting vowels in a string is done by traversing the characters until the null character and testing each one.

A palindrome check compares characters from both ends of the string moving towards the middle.

`fgets` reads a line of text safely because it limits the number of characters stored in the buffer.

Text processing tasks such as word counting split a line into tokens separated by spaces or punctuation.

4. Practice activities

Practice 1: characters: Write and test a short program that illustrates this rule: The functions in `ctype.h` such as `toupper`, `tolower`, `isalpha` and `isdigit` classify and convert individual characters.

Input: at least two test cases of your choice → Expected output: output that matches your hand-traced result

Practice 2: strcmp: Write and test a short program that illustrates this rule: The functions `strlen`, `strcpy`, `strcat` and `strcmp` are declared in the header file `string.h`.

Input: at least two test cases of your choice → Expected output: output that matches your hand-traced result

5. Why this was recommended

The knowledge-tracing model estimates your mastery of 'String Manipulation and Text Processing' at 50%, below the OBE target. In your latest assessment you answered 0 of 3 questions on this concept correctly; for example you missed "Which header file declares `strlen`, `strcpy` and `strcmp`?". The plan focuses on the topics that concept's questions depend on.

Self-check: tick when you can explain it without notes

- characters
- strcmp
- compares
- buffer

Priority 2: Operators, Pointers and Control Structures

CO2 · Mastery 51% · exam score 87% · level Intermediate

1. What you will be able to do

- Explain the key ideas of Operators, Pointers and Control Structures.
- Apply statement correctly in small programs.
- Identify common mistakes involving evaluates.

2. Topics to revise

1. statement

- The if-else statement selects between two paths, and the switch statement selects among many constant cases.
- A break statement is needed at the end of each switch case to prevent execution from falling through to the next case.

2. evaluates

- Integer division truncates the fractional part, so $7 / 2$ evaluates to 3 while $7 \% 2$ evaluates to 1.

3. while

- The for, while and do-while loops repeat a block of statements; a do-while loop always executes its body at least once.

4. address

- A pointer is a variable that stores the memory address of another variable.
- The address-of operator `&` gives the address of a variable, and the dereference operator `*` accesses the value at an address.

3. Explanation

Integer division truncates the fractional part, so $7 / 2$ evaluates to 3 while $7 \% 2$ evaluates to 1.

Relational operators compare two values and produce 1 for true and 0 for false. Logical operators such as `&&` and `||` use short-circuit evaluation, so the right operand may not be evaluated.

Modifying a variable twice in one expression without a sequence point, as in `x += x++`, is undefined behaviour.

Reading material from your syllabus

These are the parts of the course notes that cover this concept. Read them before you try the practice activities.

Unit 2: Operators, Pointers and Control Structures

Arithmetic operators in C include addition, subtraction, multiplication, division and the modulus operator.

Integer division truncates the fractional part, so $7 / 2$ evaluates to 3 while $7 \% 2$ evaluates to 1.

Relational operators compare two values and produce 1 for true and 0 for false.

Logical operators such as `&&` and `||` use short-circuit evaluation, so the right operand may not be evaluated.

Modifying a variable twice in one expression without a sequence point, as in `x += x++`, is undefined behaviour.

The if-else statement selects between two paths, and the switch statement selects among many constant cases.

A break statement is needed at the end of each switch case to prevent execution from falling through to the next case.

The for, while and do-while loops repeat a block of statements; a do-while loop always executes its body at least once.

A pointer is a variable that stores the memory address of another variable.

The for, while and do-while loops repeat a block of statements; a do-while loop always executes its body at least once.

A pointer is a variable that stores the memory address of another variable.

The address-of operator `&` gives the address of a variable, and the dereference operator `*` accesses the value at an address.

Pointer arithmetic is scaled by the size of the pointed-to type, so incrementing an int pointer advances by `sizeof(int)` bytes.

Dereferencing a NULL or uninitialised pointer causes undefined behaviour and usually a crash.

Passing a pointer to a function lets the function modify the caller's variable, which is called call by reference.

4. Practice activities

Practice 1: statement: Write and test a short program that illustrates this rule: The if-else statement selects between two paths, and the switch statement selects among many constant cases.

Input: at least two test cases of your choice → Expected output: output that matches your hand-traced result

Practice 2: evaluates: Write and test a short program that illustrates this rule: Integer division truncates the fractional part, so $7 / 2$ evaluates to 3 while $7 \% 2$ evaluates to 1.

Input: at least two test cases of your choice → Expected output: output that matches your hand-traced result

5. Why this was recommended

The knowledge-tracing model estimates your mastery of 'Operators, Pointers and Control Structures' at 51%, below the OBE target. In your latest assessment you answered 3 of 3 questions on this concept correctly. The plan focuses on the topics that concept's questions depend on.

Self-check: tick when you can explain it without notes

- statement
- evaluates
- while
- address

Priority 3: Fundamentals and System Architecture

CO1 · Mastery 58% · exam score 100% · level Intermediate

1. What you will be able to do

- Explain the key ideas of Fundamentals and System Architecture.
- Apply instruction correctly in small programs.
- Identify common mistakes involving processed.

2. Topics to revise

1. instruction

- In the von Neumann architecture both instructions and data are stored in the same main memory.
- The CPU repeatedly fetches an instruction from memory, decodes it and executes it, which is called the instruction cycle.

2. processed

- Registers are small, very fast storage locations inside the CPU that hold values currently being processed.
- A C source file is first processed by the preprocessor, which expands macros and includes header files.

3. source

- The compiler translates the preprocessed source code into assembly or object code for the target machine.

4. object

- The linker combines object files and library code into a single executable program.

3. Explanation

In the von Neumann architecture both instructions and data are stored in the same main memory. The CPU repeatedly fetches an instruction from memory, decodes it and executes it, which is called the instruction cycle. Registers are small, very fast storage locations inside the CPU that hold values currently being processed. A C source file is first processed by the preprocessor, which expands macros and includes header files.

Reading material from your syllabus

These are the parts of the course notes that cover this concept. Read them before you try the practice activities.

Unit 1: Fundamentals and System Architecture

A computer system consists of the CPU, main memory, secondary storage and input/output devices connected by buses.

In the von Neumann architecture both instructions and data are stored in the same main memory. The CPU repeatedly fetches an instruction from memory, decodes it and executes it, which is called the instruction cycle.

Registers are small, very fast storage locations inside the CPU that hold values currently being processed.

A C source file is first processed by the preprocessor, which expands macros and includes header files.

The compiler translates the preprocessed source code into assembly or object code for the target machine.

The linker combines object files and library code into a single executable program.

A program that compiles successfully can still fail at link time when a function is declared but never defined.

A program that compiles successfully can still fail at link time when a function is declared but never defined.

Every C program starts executing from the main function, which returns an integer status to the operating system.

The basic data types in C are char, int, float and double, and their sizes depend on the platform.

Variables must be declared with a type before they are used, and uninitialised local variables contain garbage values.

Standard input and output are performed with the scanf and printf functions declared in stdio.h.

4. Practice activities

Practice 1: instruction: Write and test a short program that illustrates this rule: In the von Neumann architecture both instructions and data are stored in the same main memory.

Input: at least two test cases of your choice → Expected output: output that matches your hand-traced result

Practice 2: processed: Write and test a short program that illustrates this rule: Registers are small, very fast storage locations inside the CPU that hold values currently being processed.

Input: at least two test cases of your choice → Expected output: output that matches your hand-traced result

5. Why this was recommended

The knowledge-tracing model estimates your mastery of 'Fundamentals and System Architecture' at 58%, below the OBE target. In your latest assessment you answered 3 of 3 questions on this concept correctly. The plan focuses on the topics that concept's questions depend on.

Self-check: tick when you can explain it without notes

- instruction
- processed
- source
- object

Priority 4: Data Structures, Memory Management, and File Handling

CO5 · Mastery 63% · exam score 61% · level Intermediate

1. What you will be able to do

- Explain the key ideas of Data Structures, Memory Management, and File Handling.
- Apply allocated correctly in small programs.
- Identify common mistakes involving node.

2. Topics to revise

1. **allocated**

- Local variables are allocated on the stack and are released automatically when the function returns.
- Dynamic memory is allocated on the heap with malloc or calloc and must be released explicitly with free.

2. **node**

- A singly linked list is a sequence of nodes where each node stores data and a pointer to the next node.
- Inserting at the head of a linked list creates a new node, points it to the current head and makes it the new head.

3. **creates**

- The mode "r" opens a file for reading, "w" creates or truncates a file for writing and "a" appends to a file.

4. **fopen**

- Files are opened with fopen, which returns a FILE pointer, and closed with fclose.
- Always check that fopen did not return NULL before reading from or writing to a file.

3. Explanation

Members of a structure are accessed with the dot operator, and through a pointer with the arrow operator. Local variables are allocated on the stack and are released automatically when the function returns. Dynamic memory is allocated on the heap with malloc or calloc and must be released explicitly with free. A memory leak occurs when allocated memory is no longer referenced but has not been freed.

Reading material from your syllabus

These are the parts of the course notes that cover this concept. Read them before you try the practice activities.

Unit 5: Data Structures, Memory Management, and File Handling

A structure groups related variables of different types under one name, for example a student record with a name and marks.

Members of a structure are accessed with the dot operator, and through a pointer with the arrow operator.

Local variables are allocated on the stack and are released automatically when the function returns.

Dynamic memory is allocated on the heap with malloc or calloc and must be released explicitly with free.

malloc returns NULL when the allocation fails, so the result must always be checked before use.

A memory leak occurs when allocated memory is no longer referenced but has not been freed.

A singly linked list is a sequence of nodes where each node stores data and a pointer to the next node.

Inserting at the head of a linked list creates a new node, points it to the current head and makes it the new head.

Files are opened with fopen, which returns a FILE pointer, and closed with fclose.

Inserting at the head of a linked list creates a new node, points it to the current head and makes it the new head.

Files are opened with fopen, which returns a FILE pointer, and closed with fclose.

The mode "r" opens a file for reading, "w" creates or truncates a file for writing and "a" appends to a file.

fprintf and fscanf perform formatted output and input on files, while fread and fwrite handle binary data.

Always check that fopen did not return NULL before reading from or writing to a file.

4. Practice activities

Practice 1: allocated: Write and test a short program that illustrates this rule: Local variables are allocated on the stack and are released automatically when the function returns.

Input: at least two test cases of your choice → Expected output: output that matches your hand-traced result

Practice 2: node: Write and test a short program that illustrates this rule: A singly linked list is a sequence of nodes where each node stores data and a pointer to the next node.

Input: at least two test cases of your choice → Expected output: output that matches your hand-traced result

5. Why this was recommended

The knowledge-tracing model estimates your mastery of 'Data Structures, Memory Management, and File Handling' at 63%, below the OBE target. In your latest assessment you answered 3 of 3 questions on this concept correctly. The plan focuses on the topics that concept's questions depend on.

Self-check: tick when you can explain it without notes

- allocated
- node
- creates
- fopen